Architecture Write-Up

Agentic PR Review & Release Notes Pipeline — Capstone Project

1. Workflow Overview

This system automates a real engineering workflow: reviewing a pull request for bugs, security issues, and silently breaking changes, then drafting a release note. It targets an engineering manager or tech lead who bottlenecks on manual review. The pipeline combines three specialized LLM agents (a planner, a reviewer, and a release-manager), one deterministic scoring step, MCP-backed persistent storage and retrieval, and a hard human checkpoint for any risky change. Full acceptance criteria, failure modes, and delivery-path details are in the project PRD.

Delivery path: job-seeker / no-deployment. The pipeline runs against real, merged pull requests from two public repositories (chalk/chalk and date-fns/date-fns) rather than live production traffic. A small set of these real PRs were modified to include one intentional, documented bug each, forming a ground-truth set used throughout evaluation.

2. System Architecture

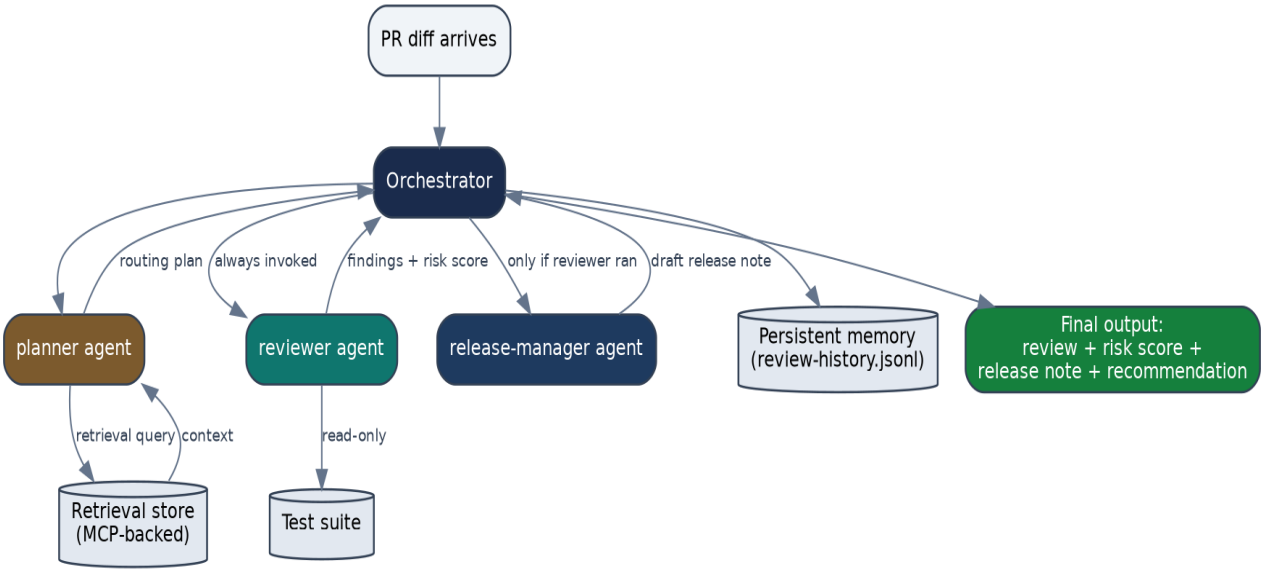

Figure 1: Orchestration flow. Colored nodes are LLM agents; cylinders are persistent/external data stores.

The orchestrator is the only component with authority to declare a run complete. It always invokes the planner first; the planner must always hand off to the reviewer (this is a structural rule, not a suggestion — see Section 6); the release-manager only runs after the reviewer has produced findings, and reads those findings to decide how to frame its note.

3. Agent Roles

Agent	Purpose	Key Constraint
planner	Parses the diff, retrieves relevant prior context via the retrieval store, produces a routing plan.	No review or write authority — routing only. Cannot skip invoking the reviewer.

Agent	Purpose	Key Constraint
reviewer	Reads the diff and produces grounded findings (file, line, issue, severity).	Read-only. No git write/merge, no secrets, no network beyond MCP tools.
release-manager	Drafts a one-line release note from the diff and the reviewer's findings.	Draft-only. Must not imply safety for a change the reviewer flagged as high/critical severity.

Two supporting skills are shared across agents rather than duplicated: **diff-parsing** (used by both planner and reviewer, so both parse diffs identically) and **risk-scoring** (used by the reviewer; see Section 4 — this one was converted from an agent-invoked skill to deterministic code).

4. Deterministic vs. Agent Steps

Not every step in this pipeline is an LLM call. A full agent-vs-deterministic-vs-human classification of every pipeline step is documented separately (decision matrix); the most consequential conversion is summarized here.

Risk-scoring: converted from agent reasoning to code

The mapping from a reviewer's findings (a list of severities) to an overall recommendation was originally designed as a skill the reviewer agent would apply via its own reasoning on every call. On inspection, the mapping rules were fully specified with no genuine ambiguity left (e.g., any critical finding → escalate to a human) — so it was rewritten as a pure Python function.

Metric	Before (LLM reasoning, estimated)	After (deterministic code, measured)
Latency	Typical LLM call range (not directly timed in this project)	0.0011 ms/call
Cost per call	Non-zero (API tokens)	$0
Consistency	Not guaranteed run-to-run	Guaranteed — 9/9 rule-branch tests pass every run

This conversion directly targets a measured weakness: in the pre-agent human baseline, reviewers scored 1 out of 2 on stating severity/recommended action consistently, across every seeded-bug case tested. Full reasoning, three rejected alternatives (each with cited evidence), and open risks are documented in ADR-001.

5. Human Checkpoints

No agent in this pipeline has git write, merge, or credential access — full stop. Beyond that hard boundary, the deterministic risk-scoring rules force specific situations to a human:

- Any **critical** or **high** severity finding → **escalate_to_human**, never auto-approved.
- A calibration log entry (a change to routing, prompts, or tool grants made because of eval evidence) requires human promotion from a proposals file — no agent can write directly to the calibration log.
- If the reviewer's LLM call is unavailable (timeout, missing credentials, retries exhausted), the orchestrator fails *closed*: it escalates to a human rather than silently approving the change.

6. MCP Boundaries & Role-to-Tool Access

Two local MCP servers back this pipeline, communicating over stdio JSON-RPC. Both independently enforce a role-based tool allow-list on every call — this is not just a config file the orchestrator is expected to respect; each server checks the caller's declared role itself and rejects unauthorized calls with a specific error.

Role	pr-review-storage	pr-review-retrieval	Git write / Secrets
orchestrator	get_review_history, put_review_record, get_calibration_log	search_context, index_stats	None
planner	get_review_history, get_calibration_log	search_context, index_stats	None
reviewer	get_review_history, get_calibration_log	search_context, index_stats	None
release-manager	None	None	None

This table is the single source of truth, cross-referenced against three places: the orchestration diagram's routing rules, each MCP server's own `TOOL_GRANTS` dict, and a CI policy test (Section 9) that fails the build if any of these three drift apart.

7. Storage & Retrieval

Persistent storage

Review history and a calibration log are stored as append-only JSON Lines files, schema-validated on every write — malformed records are rejected, not silently stored, so the audit trail stays trustworthy. Full diffs and secrets are explicitly never stored in memory; only summaries.

Retrieval

The retrieval server indexes every PR diff at the per-changed-file level using TF-IDF, with an explicit relevance floor: a query that doesn't clear the threshold returns an empty result rather than a fabricated weak match. A 6-query ground-truth retrieval test (5 positive, 1 negative/false-positive check) was run against the real server: **4/6 hit**, with one honest miss (a conceptual query with no lexical overlap — the relevance floor correctly returned nothing) and one confirmed false positive (logged as a calibration proposal for a stricter relevance threshold).

8. CI/CD Integration

A GitHub Actions workflow runs on any pull request touching agents, skills, MCP servers, evals, or governance docs — the exact file categories the project requires eval-gated merges for. Four jobs run:

- **Policy checks** — asserts the real `TOOL_GRANTS` code matches the documented governance policy, and functionally verifies a denied role really gets denied.
- **Eval checks** — runs deterministic checks (schema validity, grounding, seeded-bug detection, false-positive control) against sample reviewer output.
- **Deterministic conversion check** — runs the risk-scoring self-test (all 9 rule branches).
- **Retrieval self-test** — informational, tracks the 4/6 ground-truth hit rate as a regression baseline.

9. Governance Enforcement: A Real Drill

Rather than only assert that governance is enforced, a real permission was deliberately revoked in the storage server's code (removing the reviewer's calibration-log read access) to test whether the safety net actually catches drift.

Stage	Result
Before	5/5 policy tests pass
During (permission revoked)	4/5 pass — test_storage_grants_match_policy fails with an exact, specific diff between actual and expected TOOL_GRANTS
Downstream check	A live call to the affected tool from the reviewer role confirmed real breakage: PermissionError, not a silent failure
After rollback	5/5 policy tests pass again; the live tool call confirmed working again

This is real, captured test output, not a hypothetical — full transcript in the tool-evolution drill report.

10. Reliability & Cost Controls

Control	Verification
Per-call timeout (30s)	Code-level guarantee on both LLM and MCP subprocess calls
Retry with backoff (2 retries)	Implemented in the LLM call path; each attempt logged
Fail-closed fallback	Directly tested: pipeline runs with no API key present correctly returned escalated_fallback rather than a fabricated review, after the planner's real MCP call succeeded first
Max-iteration guard (5)	Code-level guarantee against runaway loops
Per-call / per-workflow token budgets	Directly tested: both budget types correctly raised BudgetExceededError when exceeded

11. Known Limitations

- Full end-to-end runs with automated LLM API calls require a live API key not configured in the development environment used for this write-up; the pipeline's control flow, deterministic steps, and MCP integration are proven independent of that key, and the reviewer's judgment step was validated directly (Claude performing the review reasoning in-session) against the full holdout set.
- Retrieval is lexical (TF-IDF), not semantic — it cannot match conceptual queries with no vocabulary overlap to relevant content, a known and documented limitation.
- Ground-truth and holdout sets are small (single digits of PRs) — sufficient to catch obvious silent failures, not a statistically rigorous benchmark.